TRABAJO DE CBD

FitLog API

Realizado por

Miguel Regidor García
Pablo Arrabal Hermoso

Titulación de
Grado en Ingeniería Informática - Ingeniería del Software

En el departamento de
Lenguajes y Sistemas Informáticos

Convocatoria de Evaluación Ordinaria, curso 2025/26

Índice general

1 Introducción **1**
1.1. Contexto y Estado del Arte 1
1.2. Motivación y Justificación 1
1.3. Objetivos 2
1.4. Alcance y Limitaciones 2

2 Análisis de requisitos **3**
2.1. Descripción de alto nivel de los requisitos 3
2.2. Requisitos funcionales 3

3 Arquitectura de datos **5**
3.1. Modelado de datos 5
3.2. Justificación del Modelo NoSQL 6
3.3. Validaciones y restricciones 6

4 Implementación **7**
4.1. Organización del código 7
4.2. Operaciones CRUD sobre los recursos 8
4.3. Mecanismos avanzados de Mongoose y decisiones arquitectónicas . 8
4.3.1. Middleware (Hooks) para la seguridad en operaciones de escritura 8
4.3.2. Propiedades virtuales y agregación relacional 9
4.3.3. Operaciones atómicas para la concurrencia e integridad . . . 9
4.3.4. Gestión de relaciones y optimización de consultas 9
4.3.5. Defensa en profundidad: Validación en dos capas y asincronía 10

5 Manual de instalación **11**
5.1. Versión desplegada 11
5.2. Instalación local 11

6 Documentación de la API **12**
6.1. Endpoints de autenticación 12
6.2. Gestión de usuarios 12
6.3. Gestión de ejercicios 13
6.4. Gestión de rutinas 13
6.5. Gestión de sesiones 14
6.6. Funcionalidad social 14

7 Manual de usuario **15**
7.1. Registro e inicio de sesión 15
7.2. Gestión de rutinas y sesiones 15

7.3. Funcionalidad social 15

8 Diseño de la interfaz de usuario 16

9 Conclusiones 18

A Autodeclaración del uso de IA 19

B Desglose de tiempo 20

Índice de figuras

8.1. Pantalla de los ejercicios. 16
8.2. Pantalla de creación de rutinas. 16
8.3. Pantalla de creación de sesiones. 17
8.4. Pantalla de amigos. 17

Índice de tablas

3.1. Colecciones principales del modelo de datos 5

6.1. Endpoints de autenticación 12
6.2. Endpoints de gestión de usuarios 12
6.3. Endpoints de gestión de ejercicios 13
6.4. Endpoints de gestión de rutinas 13
6.5. Endpoints de gestión de sesiones 14
6.6. Endpoints de funcionalidad social 14

B.1. Desglose de tiempo por tarea y responsable 20

1. Introducción

1.1. Contexto y Estado del Arte

En la última década, el sector del fitness ha experimentado una transformación digital sin precedentes. La monitorización del entrenamiento de fuerza ha pasado de los diarios en papel a ecosistemas digitales complejos que permiten el análisis del rendimiento en tiempo real. Este fenómeno, de "datificación del entrenamiento", se basa en el registro continuo de información como el volumen de carga, la intensidad, la frecuencia y los tiempos de recuperación.

En el mercado actual, existen soluciones consolidadas como Hevy, Strong o FitNotes. Aplicaciones como Hevy, por ejemplo, han tenido éxito gracias a su enfoque social y una interfaz de usuario intuitiva que facilita el registro de series y repeticiones. Sin embargo, estas herramientas presentan ciertas limitaciones:

- **Rigidez en el modelo de datos:** a menudo es difícil registrar métricas personalizadas o variabilidad entre ejercicios que necesitan diferentes tipos de material (por ejemplo, mancuernas vs. gomas de resistencia).
- **Monetización y acceso a datos:** muchas funcionalidades de análisis avanzado están bloqueadas bajo modelos de suscripción, lo que limita la capacidad del usuario para realizar un seguimiento exhaustivo de su propia progresión histórica sin restricciones.

1.2. Motivación y Justificación

La elección de este proyecto nace de una confluencia entre la formación académica en gestión de bases de datos y una motivación personal compartida por los integrantes del equipo. Como usuarios habituales del gimnasio y entusiastas de la cultura del fitness, hemos identificado que, aunque las aplicaciones comerciales suplen la mayoría de las necesidades básicas, ninguna ofrece una solución integral que se adapte al 100 % a un perfil de usuario que busca un control total.

Ante este "hueco" de funcionalidad, decidimos desarrollar una API que no solo sirva como ejercicio académico, sino que se convierta en una herramienta real de uso personal. Además, para darle el uso real que pretendemos desarrollar, se ha incluido una interfaz de usuario.

Justificación Técnica: Para este propósito, hemos seleccionado un stack tecnológico moderno compuesto por Node.js, Express y MongoDB (con Mongoose). La elección de una base de datos NoSQL como MongoDB es fundamental por las siguientes razones:

- **Flexibilidad de esquema:** el entrenamiento es heterogéneo. Un ejercicio de powerlifting (basado en series y repeticiones) requiere datos muy distintos a un entrenamiento de resistencia o una sesión de movilidad. MongoDB nos permite almacenar documentos con estructuras variables sin romper la integridad del sistema.
- **Escalabilidad:** pensando en un futuro uso o en la incorporación de una app móvil y su posible lanzamiento, MongoDB resulta un factor de gran valor.

1.3. Objetivos

El proyecto se articula en torno a los siguientes objetivos:

Objetivo general: Diseñar e implementar una API REST funcional y escalable para la gestión de sistemas de entrenamiento, garantizando la persistencia de datos complejos y la integridad de la lógica de negocio de este dominio.

Objetivos Específicos:

- Modelado de datos avanzado: desarrollar esquemas en Mongoose que permitan representar de forma jerárquica y eficiente la relación entre usuarios, rutinas, ejercicios y registros de sesión.
- Validación estricta: Implementar capas de validación que aseguren que los datos introducidos (pesos, repeticiones, fechas) sean coherentes y útiles para un análisis posterior.
- Preparación para el consumo externo: documentar y desplegar la API de manera que pueda ser consumida de forma sencilla por cualquier frontend o aplicación móvil en fases posteriores.

1.4. Alcance y Limitaciones

El alcance de este proyecto abarca el desarrollo de una solución integral de gestión de entrenamiento, compuesta por una API REST y una interfaz de usuario web funcional. Aunque el sistema será plenamente funcional en su versión web, el desarrollo de una aplicación móvil nativa queda fuera del alcance de este periodo académico debido a su complejidad técnica adicional.

2. Análisis de requisitos

2.1. Descripción de alto nivel de los requisitos

La aplicación debe permitir la gestión básica de entrenamientos de gimnasio, rutinas y ejercicios, con una estructura que facilite el registro progresivo de sesiones y el histórico de actividad del usuario. La primera versión cubrirá la creación y edición de usuarios, ejercicios, rutinas y sesiones de entrenamiento, permitiendo definir entrenos compuestos por ejercicios ordenados y series asociadas con repeticiones, peso y observaciones.

Además, el sistema tendrá una capa de autenticación y gestión de cuentas, de forma que cada usuario pueda registrarse, iniciar sesión, editar su perfil y mantener sus datos personales. A partir de ahí, la aplicación también incluirá relaciones sociales entre usuarios, como añadir amigos, y consultar su actividad, incluyendo la visualización de entrenamientos previos compartidos.

2.2. Requisitos funcionales

- El sistema debe permitir crear, consultar, modificar y eliminar usuarios.
- El sistema debe permitir crear, consultar y eliminar ejercicios.
- El sistema debe permitir crear, consultar, modificar y eliminar rutinas.
- El sistema debe permitir crear sesiones de entrenamiento a partir de una rutina o manualmente y añadir los ejercicios uno a uno.
- El sistema debe permitir registrar ejercicios dentro de una sesión de entrenamiento.
- El sistema debe permitir registrar series asociadas a cada ejercicio, con repeticiones, peso y datos complementarios.
- El sistema debe permitir marcar una sesión como finalizada y guardarla en el histórico.
- El sistema debe permitir consultar sesiones previas asociadas a un usuario.
- El sistema debe permitir guardar rutinas como un conjunto ordenados de ejercicios.
- El sistema debe permitir registrar y actualizar información básica del perfil del usuario.
- El sistema debe permitir crear cuentas de usuario mediante un proceso de registro.

- El sistema debe permitir autenticación de usuarios mediante inicio de sesión.
- El sistema debe permitir que cada usuario acceda a sus propios entrenamientos y rutinas.
- El sistema debe permitir añadir amigos con otros usuarios.
- El sistema debe permitir consultar el perfil y los entrenamientos compartidos de otros usuarios, si existe relación de amistad.
- El sistema debe validar los datos introducidos en cada operación para evitar información inconsistente.
- El sistema debe permitir configurar un temporizador tras completar una serie para monitorizar el tiempo entre series.

3. Arquitectura de datos

3.1. Modelado de datos

El diseño de la persistencia se basa en un modelo no relacional orientado a documentos con MongoDB, el núcleo del sistema se define a través de cuatro colecciones independientes:

Entidad	Descripción	Relaciones y Propósito
User	Perfil de usuario y datos de acceso.	Almacena preferencias de entrenamiento y configuraciones de seguridad.
Exercise	Ejercicios cargados en la aplicación.	Entidad principal que incluye datos (músculos, equipo, dificultad, etc.).
Routine	Plan de entrenamiento predefinido.	Estructura estática que sirve de plantilla para comenzar sesiones de entrenamiento con una serie de ejercicios ya predefinidos y con un orden.
WorkoutSession	Instancia de ejecución de un entrenamiento.	Documento dinámico que registra el volumen de carga y el progreso histórico.

Tabla 3.1: Colecciones principales del modelo de datos

Para evitar el coste de consultas cruzadas en tiempo de ejecución y aplicar los principios teóricos, se ha optado por:

- **Composición de rutinas (RoutineExercise):** en lugar de descomponer cada rutina en múltiples colecciones auxiliares, cada documento Routine incluye un array de subdocumentos con el orden de ejecución, las series/repeticiones objetivo y parámetros de planificación de cada ejercicio.
- **Registro de ejecución (Entry y PerformedSet):** cada WorkoutSession encapsula su propio estado de entrenamiento, incluyendo los ejercicios planificados y los resultados reales de cada serie. El subdocumento PerformedSet almacena los datos de ejecución por serie (repeticiones, peso, descanso y estado de completado).

Este diseño reduce la complejidad de lectura y escritura para los flujos críticos (crear rutina, iniciar sesión y registrar progreso), manteniendo la coherencia de los datos históricos dentro de cada sesión.

3.2. Justificación del Modelo NoSQL

La elección de MongoDB frente a un sistema relacional tradicional se justifica por la naturaleza heterogénea de los datos. En un sistema de entrenamiento existen elementos muy estructurados, como el usuario o el catálogo de ejercicios, pero también existen estructuras altamente variables, como una sesión de entrenamiento, que puede contener distintos ejercicios, distintas series, tiempos de descanso y observaciones.

En una base de datos relacional, este tipo de información suele repartirse en múltiples tablas intermedias, lo que obliga a realizar un número elevado de joins para reconstruir una sesión completa. En cambio, MongoDB permite agrupar los datos relacionados dentro de un mismo documento, simplificando tanto la escritura como la lectura de información compleja.

3.3. Validaciones y restricciones

La integridad de la información se asegura mediante validaciones en dos niveles. Por un lado, Mongoose define restricciones en los esquemas, como campos obligatorios, rangos numéricos, longitudes mínimas y máximas, valores enumerados y referencias válidas entre documentos. Por otro lado, se utiliza Zod para validar la entrada de datos antes de que llegue al controlador, lo que evita que solicitudes incorrectas entren en la lógica de negocio.

Por ejemplo, no se permite crear una rutina sin nombre, una serie con repeticiones negativas ni un usuario sin credenciales válidas. Además, la contraseña del usuario se cifra antes de almacenarse, y el acceso a ciertas rutas requiere autenticación mediante token JWT.

4. Implementación

La aplicación se ha implementado siguiendo una arquitectura REST orientada a recursos, en la que cada entidad principal del dominio se expone mediante endpoints específicos y se manipula a través de los verbos HTTP estándar. La API devuelve respuestas en formato JSON y no guarda estado.

La implementación se organiza en capas claramente separadas. La capa de rutas define los endpoints disponibles y actúa como punto de entrada de las peticiones. La capa de controladores contiene la lógica de negocio y coordina el acceso a los modelos. La capa de modelos define la estructura de persistencia en MongoDB mediante Mongoose. Además, se incluyen middleware de autenticación, validación y tratamiento de errores, lo que mejora la seguridad y la robustez del sistema.

Esta separación por responsabilidades facilita el mantenimiento del proyecto y permite que la API pueda evolucionar de forma ordenada.

4.1. Organización del código

La API se ha diseñado a partir de recursos principales que representan las entidades del dominio. La ruta base agrupa los distintos módulos funcionales y cada uno de ellos responde a un recurso o funcionalidad concreta. Sobre estos recursos se aplican operaciones de tipo CRUD, es decir, creación, consulta, actualización y eliminación, siguiendo la semántica habitual de una API REST.

El proyecto se organiza en los siguientes directorios:

```
src/
 config/
 controllers/
 middlewares/
 services/
 models/
 routes/
 utils/
 validators/
```

Cada carpeta cumple una función específica dentro de la arquitectura. models define la estructura de los datos en MongoDB, controllers contiene la lógica de negocio, routes expone los endpoints de la API, middlewares gestiona autenticación y validación, validators comprueba la entrada de datos y utils reúne funciones auxiliares reutilizables.

4.2. Operaciones CRUD sobre los recursos

La aplicación implementa operaciones CRUD sobre los recursos principales del sistema, con especial atención al catálogo de ejercicios. En este caso, los usuarios autenticados pueden registrar nuevos ejercicios, pero su publicación queda sujeta a moderación: pasan a un estado pendiente hasta que un administrador los revisa. Una vez aprobados, los ejercicios públicos se incorporan al catálogo general y pueden ser utilizados por cualquier usuario del sistema.

En el caso de los usuarios, el sistema permite registrar una cuenta, iniciar sesión, consultar el perfil propio y actualizar o eliminar la información asociada. Las rutinas permiten guardar plantillas de entrenamiento reutilizables, mientras que las sesiones almacenan el histórico real de entrenamientos completados por el usuario. Se pueden crear sesiones contando con el esqueleto de una rutina o una sesión en blanco, en ambos casos pueden editarse los ejercicios, rutinas y series que contienen.

4.3. Mecanismos avanzados de Mongoose y decisiones arquitectónicas

En la implementación de la API, se ha diseñado una arquitectura robusta que aprovecha características avanzadas de Mongoose y Node.js para encapsular la lógica de negocio, garantizar la integridad de los datos y optimizar el rendimiento.

4.3.1. Middleware (Hooks) para la seguridad en operaciones de escritura

Los *hooks* de Mongoose permiten interceptar el ciclo de vida de los documentos para automatizar procesos. En esta API, se ha centralizado la lógica de cifrado de contraseñas mediante `bcrypt` en el modelo `User`, actuando en dos frentes críticos para evitar cualquier brecha de seguridad:

- **Inserciones estándar:** Un *hook* de tipo `pre('save')` intercepta la creación de nuevos usuarios (o modificaciones realizadas mediante `document.save()`), garantizando que el algoritmo de hasheo se aplique siempre antes de la persistencia en BBDD.
- **Elusión del middleware en actualizaciones atómicas:** Una particularidad arquitectónica crítica de Mongoose es que las funciones de actualización directa (como `findOneAndUpdate`, `updateOne`, `updateMany` o `update`) se ejecutan directamente en el motor de MongoDB. Como consecuencia, estas operaciones eluden por completo el *hook* de `save`. Si este comportamiento se ignora, la actualización de una contraseña desde el perfil de usuario terminaría almacenando la clave en texto plano en la base de datos. Para solucionar esta vulnerabilidad, se ha declarado explícitamente un segundo

hook que intercepta este grupo de operaciones, extrae el contexto de la modificación mediante `this.getUpdate()` y cifra la contraseña al vuelo antes de que la consulta alcance la base de datos, blindando así absolutamente todos los vectores de escritura.

4.3.2. Propiedades virtuales y agregación relacional

Las propiedades virtuales (*virtuals*) se han empleado para calcular métricas dinámicas sin incurrir en redundancia de datos. En el modelo `User`, campos como `totalExercises` o `activeRoutines` utilizan la configuración avanzada de Mongoose vinculando `localField` (`_id`) con `foreignField` (ej. `createdBy`) y el operador `count: true`.

Esto permite que, mediante un simple `.populate('totalExercises')`, Mongoose resuelva internamente el conteo de documentos relacionados sin necesidad de escribir complejas consultas de agregación (Aggregation Framework), manteniendo los controladores limpios y las estadísticas del panel de control siempre sincronizadas. Además, se configuró el método `toJSON` en los esquemas para eliminar dinámicamente campos internos (`_id`, `__v`, `password`) antes de enviar las respuestas al cliente.

4.3.3. Operaciones atómicas para la concurrencia e integridad

Para gestionar modificaciones en arrays y relaciones complejas, se ha priorizado el uso de operadores atómicos de MongoDB frente a la manipulación en memoria.

- **Gestión de relaciones bidireccionales:** En el servicio de amistades (`socialService`), se utilizan los operadores `$addToSet` (para evitar duplicados) y `$pull` (para eliminaciones). Estas operaciones se envuelven en un `Promise.all` para asegurar que ambos perfiles de usuario se actualicen de forma simultánea y consistente.
- **Actualización de subdocumentos:** En las sesiones de entrenamiento, se utiliza `$push` dentro de un `findOneAndUpdate` para insertar nuevos *sets* directamente en la base de datos (`entries.$.performedSets`), evitando problemas de concurrencia (condiciones de carrera) si un usuario registra múltiples series rápidamente.

4.3.4. Gestión de relaciones y optimización de consultas

En bases de datos NoSQL como MongoDB, el enfoque estándar es almacenar referencias (IDs) en lugar de duplicar información en diferentes colecciones. Para ensamblar estas relaciones de forma eficiente y evitar sobrecargar la red, la API combina dos técnicas fundamentales de Mongoose:

- **Resolución de relaciones (`populate`):** Este mecanismo se utiliza para sustituir automáticamente los identificadores referenciados por sus documentos

correspondientes. Por ejemplo, al consultar una Rutina, el uso de `populate('exercises.exercise')` indica a Mongoose que navegue dentro del array de ejercicios, busque los IDs en la colección `Exercise` y los reemplace por los objetos reales. Esto evita tener que realizar múltiples peticiones encadenadas desde el cliente para unificar los datos.

- **Proyección estricta de datos (`select`):** Para maximizar el rendimiento y la seguridad, se aplican filtros desde la base de datos para no extraer documentos completos si no es estrictamente necesario:
  - **En consultas directas:** Se aplican selectores estáticos, como la inyección de la constante `PUBLIC_USER_FIELDS` mediante `.select()` en el `userService`, para omitir datos sensibles y reducir el consumo de memoria.
  - **En relaciones (Población selectiva):** Al utilizar `populate`, se le pasa un segundo argumento de proyección (ej. `populate('entries.exercise', 'name primaryMuscles equipment')`). De este modo, la API ensambla respuestas ricas en contexto, pero filtrando la información superflua del documento anidado para mantener la carga útil (payload) lo más ligera posible.

4.3.5. Defensa en profundidad: Validación en dos capas y asincronía

La lógica de validación se ha desacoplado utilizando un enfoque de "Defensa en Profundidad"(Defense in Depth). Los datos de entrada (`req.body`) son interceptados y sanitizados en los controladores mediante esquemas de **Zod** antes de interactuar con Mongoose. Esto asegura que la capa de base de datos solo procese información limpia. Asimismo, se ha implementado un contenedor `asyncHandler` que envuelve los controladores, capturando automáticamente cualquier excepción o rechazo de promesas y derivándolo al *middleware* global de errores, eliminando la necesidad de bloques `try/catch` repetitivos y unificando el formato de respuesta (`ApiError`).

5. Manual de instalación

El repositorio del proyecto se encuentra en: https://github.com/pabloarrabalh/fitlog-api.git

5.1. Versión desplegada

Se ha desplegado una versión web de la API utilizando MongoAtlas, Render y Vercel, y esta es accesible mediante la URL:

- https://fitlog-api.vercel.app/ (frontend)
- https://fitlog-api-41it.onrender.com (backend, sin interfaz visual)

5.2. Instalación local

Para que la API funcione correctamente en local es necesario instalar MongoDB Server, disponible en https://www.mongodb.com/try/download/community. Instalando el servidor y opcionalmente MongoDB Compass. En primer lugar es necesario establecer las variables de entorno a partir del .env.example en el repositorio:

```
cp .env.example .env
```

Una vez instalado se recomienda poblar la base de datos con algunos datos de ejemplo que se han incluido mediante:

```
npm run seed:exercises
npm run seed:users
```

Posteriormente el backend se inicializa con el comando:

```
npm install
npm run dev
```

Para arrancar el frontend:

```
cd frontend
npm install
npm run dev
```

6. Documentación de la API

En este capítulo se describen los endpoints disponibles en la API, su funcionalidad y ejemplos de uso. La autenticación y la persistencia de la sesión se gestionan con JWT. Cuando el usuario inicia sesión, el backend genera un token que lo identifica sin necesidad de guardar la sesión en el servidor. Se usa JWT porque permite una API sin estado y simplifica la protección de endpoints sin depender de sesiones en memoria o en base de datos, todo esto se gestiona con los endpoints de Auth:

6.1. Endpoints de autenticación

Método	Endpoint	Descripción
POST	/auth/register	Registrar un nuevo usuario
POST	/auth/login	Iniciar sesión en una cuenta existente
GET	/auth/me	Obtener el usuario autenticado

Tabla 6.1: Endpoints de autenticación

6.2. Gestión de usuarios

Método	Endpoint	Descripción
GET	/users/me	Obtener mi perfil
PATCH	/users/me	Actualizar mi perfil
DELETE	/users/me	Eliminar mi perfil
GET	/users	Listar todos los usuarios de la aplicación
GET	/users/userId	Obtener un usuario desde su identificador

Tabla 6.2: Endpoints de gestión de usuarios

6.3. Gestión de ejercicios

Método	Endpoint	Descripción
GET	/exercises	Obtener todos los ejercicios
POST	/exercises	Crear un ejercicio
GET	/exercises/mine	Listar mis ejercicios
GET	/exercises/pending	Listar ejercicios pendientes de aprobación
GET	/exercises/exerciseId	Obtener un ejercicio con su identificador
PATCH	/exercises/exerciseId	Actualizar un ejercicio
DELETE	/exercises/exerciseId	Borrar un ejercicio
PATCH	/exercises/exerciseId/approve	Aprobar un ejercicio propuesto
PATCH	/exercises/exerciseId/reject	Rechazar un ejercicio propuesto

Tabla 6.3: Endpoints de gestión de ejercicios

6.4. Gestión de rutinas

Método	Endpoint	Descripción
GET	/routines	Obtener mis rutinas
POST	/routines	Crear una rutina
GET	/routines/routineId	Obtener una rutina por su identificador
DELETE	/routines/routineId	Borrar una rutina
PATCH	/routines/routineId	Actualizar una rutina

Tabla 6.4: Endpoints de gestión de rutinas

6.5. Gestión de sesiones

Método	Endpoint	Descripción
GET	/sessions	Obtener mis sesiones de entrenamiento
POST	/sessions	Crear una sesión de entrenamiento
GET	/sessions/sessionId	Obtener una sesión por su identificador
PATCH	/sessions/sessionId	Actualizar una sesión
DELETE	/sessions/sessionId	Borrar una sesión
DELETE	/sessions/sessionId	Cancelar una rutina que se encuentra activa
POST	/sessions/sessionId/entries/entryId/sets	Añadir series a una sesión
POST	/sessions/sessionId/complete	Terminar una sesión

Tabla 6.5: Endpoints de gestión de sesiones

6.6. Funcionalidad social

Método	Endpoint	Descripción
GET	/social/friends	Obtener mis amigos
POST	/social/friends/friendId	Añadir un amigo
DELETE	/social/friends/friendId	Eliminar un amigo
GET	/social/friends/friendId/workouts	Listar los entrenamientos de un amigo

Tabla 6.6: Endpoints de funcionalidad social

Toda la documentación de la api está disponible en: http://localhost:3000/api-docs, generando utilizando Swagger UI.

7. Manual de usuario

En esta sección se describe el funcionamiento de la aplicación desde el punto de vista del usuario final, explicando las principales funcionalidades disponibles y su uso.

7.1. Registro e inicio de sesión

Para comenzar a utilizar la aplicación, el usuario debe registrarse proporcionando sus credenciales a través de la interfaz web. Una vez completado el proceso de registro, podrá iniciar sesión y acceder a todas las funcionalidades disponibles en la plataforma.

7.2. Gestión de rutinas y sesiones

La aplicación permite a los usuarios crear rutinas personalizadas, en las que pueden añadir ejercicios, definir su orden y establecer rangos de repeticiones según sus objetivos de entrenamiento. Asimismo, los usuarios pueden proponer nuevos ejercicios; estos deberán ser previamente revisados y aprobados por un administrador antes de ser publicados y visibles para el resto de usuarios.

En cuanto a las sesiones de entrenamiento, el usuario puede iniciarlas de dos formas: a partir de una rutina previamente creada, en cuyo caso las repeticiones y parámetros se configuran automáticamente, o mediante una sesión en blanco, donde dichos valores se asignan por defecto y pueden ser modificados manualmente.

Durante una sesión activa, la interfaz proporciona acceso continuo al entrenamiento en curso, permitiendo al usuario registrar el progreso en tiempo real. Entre las funcionalidades disponibles se incluyen completar series, consultar el tiempo de descanso restante, visualizar la duración total del entrenamiento y modificar los ejercicios de la sesión (añadir, eliminar o reordenar ejercicios, así como gestionar sus series).

7.3. Funcionalidad social

La plataforma incorpora una capa social que permite a los usuarios buscar y añadir amigos. Además, es posible consultar las rutinas y sesiones públicas de otros usuarios, lo que favorece la motivación y el seguimiento del progreso dentro de la comunidad.

8. Diseño de la interfaz de usuario

Adicionalmente al desarrollo de la API, se ha incluido una interfaz de usuario para ofrecer una experiencia fluida e intuitiva al consumidor final, implementada mediante el uso de React. A continuación se presentan las vistas creadas:

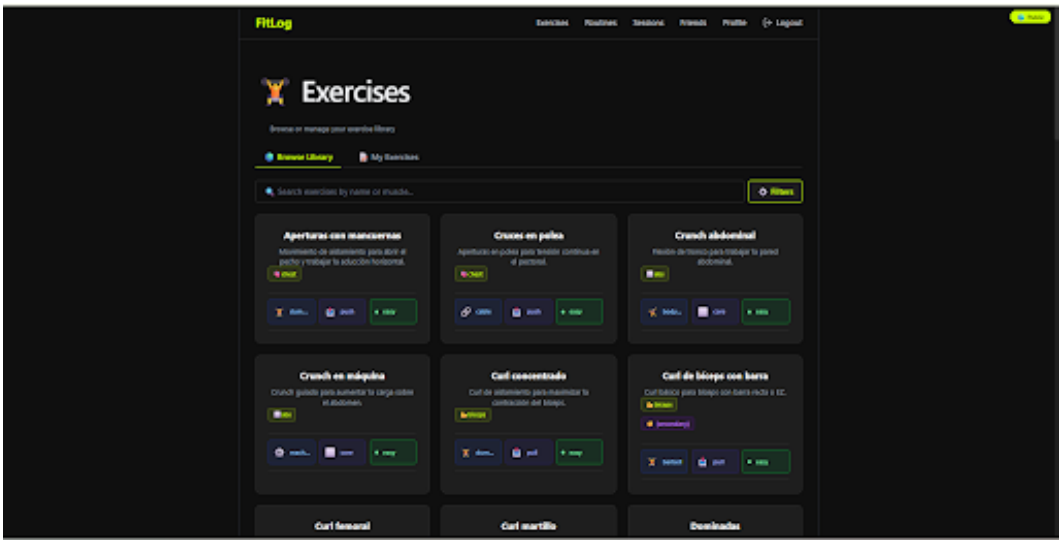

Figura 8.1: Pantalla de los ejercicios.

Para la visualización de ejercicios se ha elaborado un listado, el cual muestra información de cada ejercicio: grupo muscular principal y secundario, equipamiento necesario, dificultad y el tipo de movimiento. Adicionalmente, incluye la creación de ejercicios.

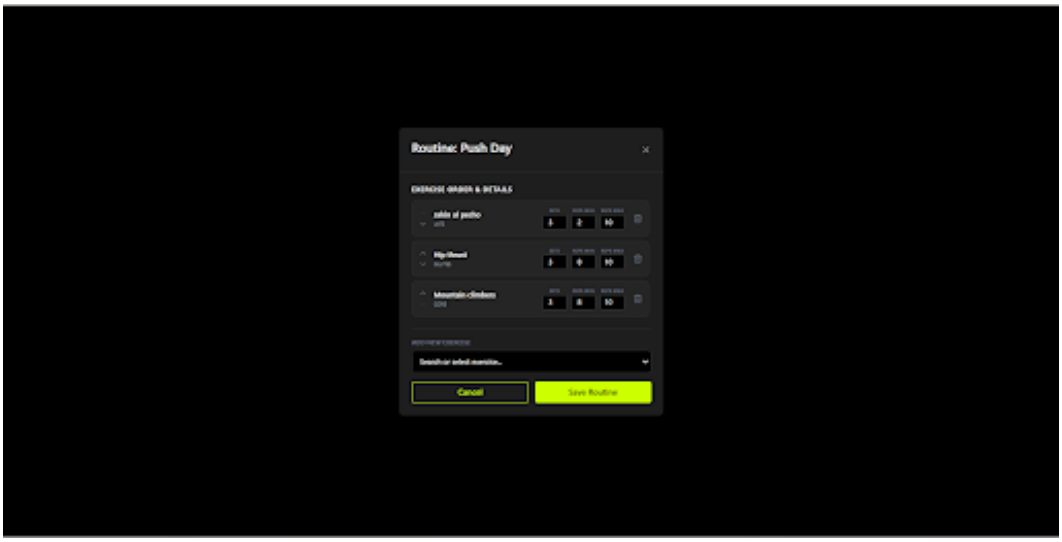

Figura 8.2: Pantalla de creación de rutinas.

Para la creación de rutinas, el usuario dispone del listado de todos los ejercicios y la posibilidad de cambiar el orden, el número de series y de repeticiones que se desea completar.

La creación de sesiones como se ha explicado previamente se distingue en dos tipos, en primer lugar, rutinas predefinidas en cuyo caso, se toma de ahí el valor de repeticiones y series, y entrenamientos vacíos, para los que se han definido valores por defecto.

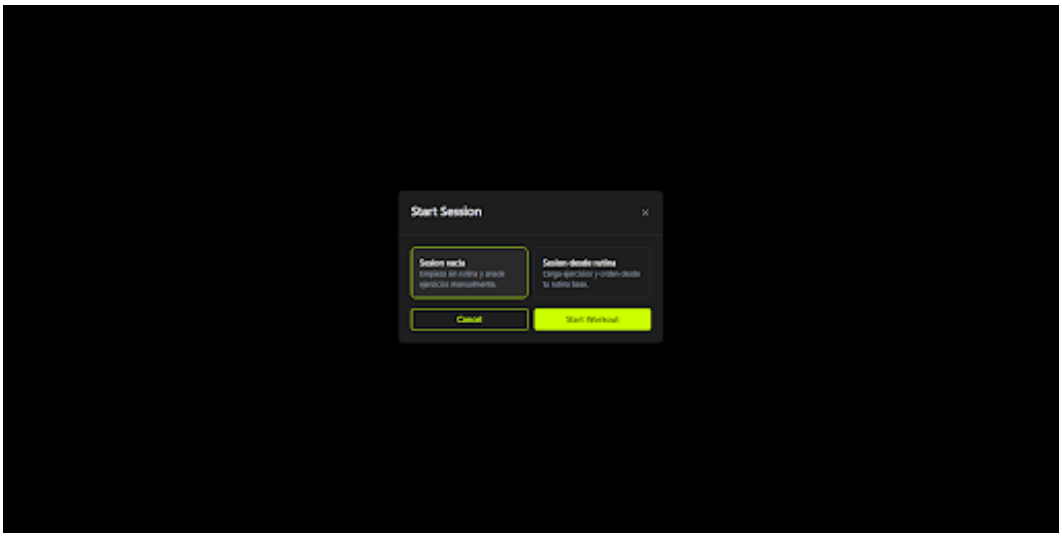

Figura 8.3: Pantalla de creación de sesiones.

Para la capa social se ha añadido una vista, con la posibilidad de buscar usuarios, añadirlos como amigos y ver sus sesiones de entrenamiento previas.

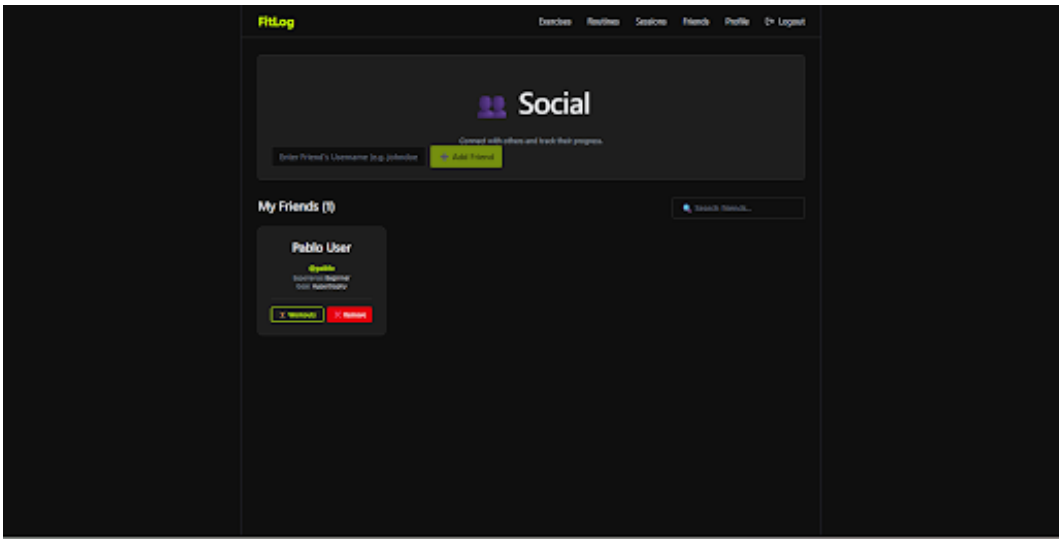

Figura 8.4: Pantalla de amigos.

9. Conclusiones

El proyecto ha permitido desarrollar una API REST funcional para la gestión de entrenamientos, cumpliendo los objetivos propuestos. Se ha conseguido implementar un sistema completo con usuarios, ejercicios, rutinas, sesiones y funcionalidad social.

La elección de MongoDB ha sido adecuada, ya que se adapta bien a la estructura variable de los datos de entrenamiento y simplifica el modelado mediante documentos. La arquitectura en capas (rutas, controladores, modelos y middleware) ha facilitado la organización del código y su mantenimiento.

Además, el proyecto continuará tras su entrega, ya que consideramos que se ha desarrollado un producto útil para cubrir el "gap" detectado en las soluciones actuales. Nos hubiera gustado desarrollar una interfaz más completa e integrar una aplicación móvil, pero por limitaciones de tiempo estas mejoras quedan fuera del alcance de esta entrega.

Por otro lado, el principal problema encontrado durante el desarrollo ha sido la falta de APIs públicas de las que obtener los datos de los ejercicios, todas aquellas consultadas no proporcionaban el nivel de detalle suficiente.

En conjunto, el proyecto cumple su función como sistema backend completo y escalable, y sirve como base sólida para futuras ampliaciones como una app móvil o un frontend más avanzado.

A. Autodeclaración del uso de IA

Se han utilizado herramientas de inteligencia artificial generativa como ChatGPT y Gemini con la herramienta Copilot integrada en VS Code (no se ha podido compartir los chats), como soporte en ciertas tareas del proyecto, tales como:

- Generación de datos de prueba.
- Generación de la documentación con Swagger UI.
- Revisión de la expresión y redacción de la memoria.
- Dudas puntuales con alguna tecnología, en lugar de revisar la documentación oficial.
- Generación de la base del proyecto frontend.
- Refactoriaciones para optimizar el código.
- Conversión a formato Latex desde Google Docs.
- Documentación y comentarios de algunos métodos en el código.

Ejemplos de prompts utilizados:

> "Tengo este documento en un Google Docs, quiero que lo pases a tex sin cambiar nada del contenido, solo pon el formato bien"

> "Dame ejemplos de funcionalidade más avanzadas de Mongoose y cómo podríamos introudcirlo en un API de gimnasio"

> "Necesito que analices mis controladores en máxima profundidad y generes la base para un proyecto frontend en React+Vite siguiendo los mejores estándares posibles tratando de maximizar la reutilización de componentes y limpieza de código y teniendo en cuenta a la hora de hacer los estilos que mantengan un buen UX/UI"

> "Actúa como un ingeniero software senior y tu trabajo depende de este proyecto: analiza este servicio y generame un informe con posibles codesmells que sean críticos y necesarios refactorizar, junto con propuestas de posibles refactoriaciones que permitan optimizar el código o mejorar su legibilidad."

B. Desglose de tiempo

Responsable	Tarea	Id	Tiempo empleado
Miguel Regidor	Estudio previo de las tecnologías	-	1h
Pablo Arrabal	Estudio previo de las tecnologías	-	1h
Pablo Arrabal	Inicialización y configuración inicial	#1	2h
Pablo Arrabal	feat: basic crud operations	#2	3h
Miguel Regidor	feat: basic crud operations	#2	1h
Pablo Arrabal	feat: controllers v2 & validation	#3	3h
Pablo Arrabal	feat: add validators logic	#4	2h
Miguel Regidor	Elaboración de la memoria y presentación	-	3h
Pablo Arrabal	Elaboración de la memoria y presentación	-	3h
Pablo Arrabal	feat: add jwt auth and social features	#5	3h
Miguel Regidor	Refactorización del backend	-	6h
Pablo Arrabal	Refactorización del backend	-	3h
Miguel Regidor	Refactorización controladores	-	3h
Miguel Regidor	Creación de servicios	-	4h
Miguel Regidor	Frontend del proyecto	-	4h
Pablo Arrabal	Sesiones y rutinas	-	5h
Miguel Regidor	Amigos y dashboard	-	5h
Pablo Arrabal	Corrección de errores	-	3h
Miguel Regidor	Corrección de errores	-	3h
Pablo Arrabal	Maquetación en Latex	-	1h
Miguel Regidor	Revisión final y entrega	-	1h
Pablo Arrabal	Revisión final y entrega	-	1h

Tabla B.1: Desglose de tiempo por tarea y responsable